

# FractalSortCPU: Bandwidth-Efficient Compressed Radix Sort on CPU

Michael Dang'ana  
*Electrical & Computer Engineering*  
*University of Toronto*  
*Eonforge Labs*  
 Toronto, Canada  
 michael.dangana@mail.utoronto.ca

**Abstract**—Cloud database systems, particularly their middleware and query execution layers, use sorting as a core operation in query processing, indexing and join execution. Distribution-dependence and limited parallelism are key issues inherent in state-of-the-art radix sort which is preferred for large datasets due to performance advantages over comparison-based algorithms. Multi-pass bucketing, stochastic sampling and dependence graph structures are common solutions to these problems that incur the cost of data pre-processing and increased memory footprint hence they are less appropriate for large-scale workloads common in cloud environments. In-place radix sort schemes increase the number of passes as precision increases, which negatively impacts latency.

Our work solves these problems by introducing a CPU-adapted histogram compression scheme for radix sorting for arbitrary-precision keys implemented on the CPU for increased accessibility, providing state-of-the-art execution time, while limiting histogram growth. Fully parallel key-based histogram updates eliminate the need for input bucketing and data pre-processing further lowering latency, mitigating distribution-dependence and reducing complexity. With a parallelized sorting architecture utilizing SIMD-accelerated operations for low latency, the algorithm demonstrates improvement over the state-of-the-art on the CPU, GPU, and FPGA by 6x, 3x and 2.5x in bandwidth efficiency on 512MB to 32GB data sets at 16-bit precision.

## I. INTRODUCTION

As workloads grow and query demand tightens, cloud database middleware and query execution systems face large-scale sorting challenges. Sorting is a key concern in big data scenarios and is impacted by growing datasets and resource scarcity, increasing the latency, throughput and consistency requirements locally and in the cloud [43] [55]. Sorting enables big data use cases for operations such as index creation, sort-merge joins, queries, and database partitioning [15]. Radix sort algorithm is ideal for larger problem sizes compared to fast approaches based on sorting networks, merge sort, and sample sort [22] [25] [26].

State-of-the-art radix sort implementations usually suffer from *distribution dependence* where skewed distributions lead to imbalanced bucket sizes affecting efficiency due to the need to suffer degradation from larger buckets or to dynamically allocate processing resources to large buckets,

*limited parallelism* due to multiple passes required for bucketing, *linear precision-scaling latency* where extra passes are necessary to achieve high precision sorts, and *memory usage growth* where pre-processing techniques including stochastic sampling, distribution histograms and dependency graphs require data structures impacting memory usage [33] [34] [25] [35] [36] [37].

Distribution dependence refers to the varying performance of sorting algorithms based on the inputs regarding skew, partially sorted data, randomness, uniform and Gaussian distribution from a latency and memory usage perspective [46]. Sorting algorithms are usually evaluated in terms of best, average and worst case performance which are significantly impacted by input distribution [47]. Various techniques are used to mitigate pre-processing necessary to determine input distribution such as adaptive partitioning and sorting strategy adjustment enabling consistent performance across a range of input distributions [48].

Limited parallelism occurs where high-performance radix sort uses multiple passes to partially sort at lower precision and store the intermediate data in memory, with a significant probability of histogram bucket collisions between entries at each pass creating critical paths [37]. Platform bottlenecks such as limited cache and socket capacity on CPU, memory conflicts and warp divergence cause bottlenecks and core idling on GPU further limiting effective parallelism, mitigated by techniques including bucket balancing and increased memory bandwidth [49].

The precision-scaling latency trade-off is characterized by the increase in sorting latency with precision where reduced precision to achieve approximate sorting is significantly beneficial in terms of reducing latency by over 40% using approximate storage, and full precision can be achieved by iteratively sorting the partially sorted data, relevant in modern hardware algorithms where reductions in compute and memory access can lead to significant performance gains for large data sets [50].

Memory utilization in radix distribution histograms can grow exponentially with precision necessitating multiple passes to accommodate memory capacity and memory access bandwidth limits on GPUs, FPGAs and CPUs for full precision sorting [17]. Temporary buffers for histogram population, sampling and bucket balancing further increase memory demand, mitigated by techniques such as adjusting the number of keys per pass to enable near-linear precision-scaling at the cost of increased memory, cache and bandwidth usage [37] [42].

This work extends FractalSort [54], originally designed for FPGA, to a CPU-adapted implementation that introduces the concept of the fractal data structure, a sparse binary tree histogram. This enables sublinear execution time for large datasets, SIMD-enabled thread execution of histogram updates, independent key processing without input bucketing or partitioning, and amortized memory overhead for large datasets due to histogram compression and localized histogram updates. The contributions include:

- 1) A parallel sort and merge update for unsorted keys with amortized update contention.
- 2) A novel batch stream sorting algorithm for large datasets without data pre-processing or input bucketing.
- 3) A novel sparse histogram compression scheme for bandwidth-efficient merge operation with counter width tapering.
- 4) A novel bandwidth-efficient FractalSortCPUA algorithm to retrieve an array of sorted keys from the compressed histogram.

Addressing these challenges enables FractalSortCPU to offer an accessible, scalable solution for cloud servers, databases and distributed big data compute systems while lowering costs and query latency at scale [44], complementing cloud resource optimization approaches [55].

## II. RELATED WORK

Radix sorting is the state-of-the-art algorithm for large keys due to the linear complexity with implementations on both serial and parallel architectures, including least significant bit (LSB) and most significant bit (MSB) algorithms often divided into histogram, prefix sum and stable scatter phases [32] [22]. One way to determine bucket bounds is coarse-grained bucketing combined with prefix scanning that offer speed-ups on uniform distributions but suffer from distribution dependence [38] [39]. Unit processor local histograms with reduction, scanning and merge scatter phases are common on GPU, memory and cache-locality are prevalent on CPU, while streaming data paths are common on FPGA due to higher memory access speeds, with memory bandwidth, radix size and precision-scalability the primary design concerns [17] [42].

Distribution dependence mitigation strategies such as sample sort and histogram sort incur pre-processing costs including increased memory footprint, which are mitigated by dependency graphs and stochastic sampling which predict load imbalance and perform smoothing techniques while relying on costly sampling processes [35] [36] [37] [40]. Such strategies increase memory footprint and pre-processing latency depending on the distribution, with significant variance between best case and worst case performance on skewed, uniform and partially sorted data [47].

Limited parallelism due to multiple passes is mitigated by balanced buckets and increased bandwidth allocation where near-linear performance is achieved [33]. Precision-scaling latency trade-off occurs in high-precision sorting scenarios which increase the number of passes to implement in-place cache-conscious algorithms where memory is limited due to histogram prefix arrays proportional to the radix size, permutation buffers for scatter stability and dependency graph data structures for distribution tracking [17]. Increasing and reducing the per-pass radix size trades off the number of passes for memory usage, while data skew causes larger bucket size variance affecting memory and tail latency, mitigated by disaggregated radix-tree layouts which minimize I/O without fully resolving the trade-off [20].

Partitioning, sampling and merging are used together to divide data sets into buckets for efficient sorting on limited compute resources used in sample sort and histogram sort, enabling balanced partitions and parallelization [17].  $k$ -way and cache-aware merging reduce passes but become the bottleneck as  $k$  increases, and external memory or streaming benefit from larger sorted runs at the cost of large  $k$ , and extra passes trade-off bandwidth for peak memory, with sampling and partitioning enabling distribution agnosticism at the cost of memory, pre-processing and merge latency [33].

Radix sort is the state of the art for hardware-based FPGA and GPU sorting, and is affected by exponential histogram growth due to both data set size and precision, and the distribution dependence due to bucketing, making memory-bandwidth a significant limiting factor [17]. The disaggregated radix tree helps reduce bandwidth usage by eliminating redundant I/O operations [20]. FractalSort [54] addresses histogram growth on FPGA through a compressed sparse histogram achieving 2.5x bandwidth efficiency over state-of-the-art FPGA sorting algorithms. In-place radix sorts and parallel counting schemes scale poorly with 64-bit and higher precisions due to the increasing number of passes needed to increase the precision of partially sorted keys and achieve a high-precision sort [41] [42].

This work solves these limitations by using a compressed sparse histogram approach eliminating the need for costly sampling, pre-processing and input bucketing, while bounding

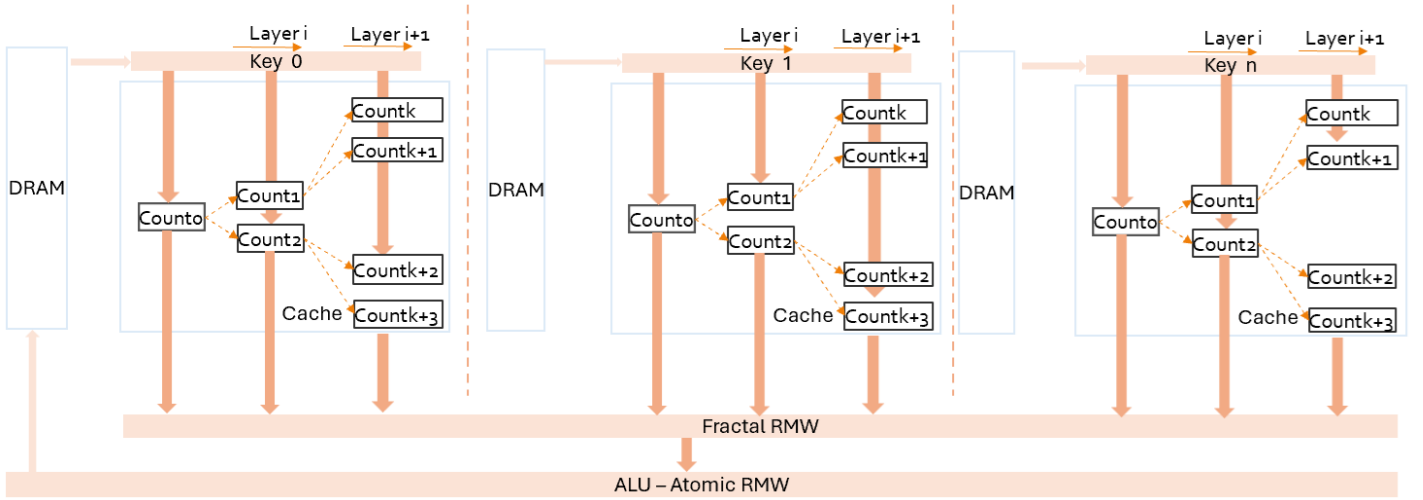


Fig. 1. FractalSortCPU Logical Layout  $p = 16$

memory usage to sublinear growth, and performing a fully parallel localized key update algorithm that minimizes data transfers, improves SIMD utilization, and ensures high bandwidth efficiency [17].

### III. SYSTEM ARCHITECTURE

The FractalSortCPU algorithm consists of key-based threads performing sequential updates to each layer of the histogram. The keys are grouped in batches and processed in parallel on CPU using SIMD vectorization via Numba JIT to maximize throughput as shown in Figure 1 [45].

#### A. Local Thread Histogram Update

Each key is mapped to a thread which updates node counter variables along the path encoded by the key bit sequence. Collisions are mitigated using atomic operations supported by the Arithmetic Logic Unit (ALU) [52].

1) *High Precision Histogram Update:* In high-precision sorting where the histogram is too large to fit in the CPU last-level cache, with  $n$  nodes and counter variables encoding  $n$  keys, the update algorithm involves two steps [53]. In the first step, a local merge on CPU where node updates are computed from key bits, and the updates are aggregated in  $O(\log(n))$  into a single local compressed tree. In the second step, the on-chip tree is transferred to memory using a parallel  $O(\log(n))$  merge operation based on a depth-first traversal from the local parent node.

#### B. Global CPU Histogram Update

The thread-based updates are merged into a single global histogram by threads co-resident on the CPU. The global

merge tree is stored both in main memory and in the CPU last-level cache (LLC) [53]. Multi-thread updates are coordinated using Atomic Read-Modify-Write (RMW) and the Fractal RMW [52].

1) *Fractal RMW:* For each key  $k$  of  $p$  bits, the histogram tree is updated by walking a root-to-leaf path determined by the key's bit sequence. At each level  $l \in \{0, \dots, L-1\}$  where  $L = \min(p, \lceil \log_2(n) \rceil)$ , the algorithm reads bit  $l$  of the key to select the left (0) or right (1) child, and atomically increments the counter at the selected node. The full path update for one key touches exactly  $L$  nodes, each requiring one Atomic RMW operation.

On x86 architecture, each atomic increment is executed in the integer ALU pipeline as a single transaction: read the counter from the cache line into a register, compute the new value, and write it back with a cache-line lock when necessary [52]. For histogram nodes in the CPU LLC, these operations are served from cache with no DRAM traffic.

Contention between threads drops exponentially with depth: at the root node all  $n$  keys contend on a single counter, but at level  $l$  the  $n$  keys are distributed across  $2^l$  nodes, giving an expected  $\frac{n}{2^l}$  updates per node. At level  $l \geq \lceil \log_2(n) \rceil$ , the expected contention falls below one, making leaf-level updates effectively contention-free. This exponential decay is what enables the fully parallel update without locks or synchronization barriers beyond the atomic instruction itself.

#### C. Batch Size Selection

The batch size parameter determines the balance between locality and parallelism, where large batch sizes reduce synchronization frequency and relatively lower the cost

---

**Algorithm 1** Insertion with Counter Width Tapering

---

**Require:** Root of histogram tree, path  $p$ , index  $i$ ,  $value$ **Ensure:** Counters updated along key path

```
1: function ADDITEM( $root, p, i, value$ )
2:    $node \leftarrow root$ 
3:   if  $node$  is null then
4:     return  $node$ 
5:   end if
6:    $node.counter = node.counter + value[i]$ 
7:   if  $p[i] \leq 0$  then
8:     if  $node.left$  is null then
9:        $node.left = NODEORTUPLE(i, value[i])$ 
10:    end if
11:     $child = node.left$ 
12:  else
13:    if  $node.right$  is null then
14:       $node.right = NODEORTUPLE(i, value[i])$ 
15:    end if
16:     $child = node.right$ 
17:  end if
18:  return ADDITEM( $child, p, i + 1, value$ )
19: end function
```

---

of thread scheduling while increasing the risk of cache overflow and cache misses. Smaller batches have low risk of cache overflow hence they enable cache reuse at the cost of increased synchronization frequency when switching between batches. Optimal batch size is tuned empirically for the target device demonstrating latency improvements of over 10% as shown in Figure 7.

#### D. Batch Counter Update

The cached histogram from a terminating batch is reused by the next batch to enable faster batch synchronization due to reduced memory transfer. This leads to a reduction in memory usage as the number of batches processed increases, as shown in Figure 8.

1) *Counter Width Compression and Overflow:* Essential to overall performance is the compression scheme involving counter width tapering in Algorithm 1. The principle exploits the reduction of significant bits on a well-balanced tree as levels increase from root to leaf, determined by the logarithm of the sub-tree size at each node  $w_{c,l} = O(\lceil \log(n_l) \rceil)$ , which over large  $n$  reduces by one at each level with leaves having approximately less than an average of two significant bits for uniformly distributed keys following Algorithm 1 and Algorithm 2.

The local and global updates have complexity  $O(\log(n))$  as shown in Table I.

#### E. Histogram Read and Write Operations

Looking up the value at an index involves depth-first path traversal of the histogram from root to leaf node. Each node has a range of indices representing the leaves in its subtree. The child selected at each node is the one containing the desired index. Traversal stops when a matching leaf node is reached. The  $getValue(index)$  hash function has  $O(\min(p, \log(n)))$  bounded latency as shown in Algorithm 2.

---

**Algorithm 2** Retrieving Items from Histogram

---

**Require:** Root of histogram tree,  $index$ ,  $defaultVal$ **Ensure:** Value at target index or  $defaultVal$  if not found

```
1: function GETITEM( $root, index, defaultVal$ )
2:    $node \leftarrow root$ 
3:   if  $node$  is null then
4:     return  $defaultVal$ 
5:   end if
6:   if  $node.counter \leq index$  then
7:      $child, val = node.left, node.value$ 
8:     return GETITEM( $child, index, val$ )
9:   else
10:     $child, val = node.right, node.value$ 
11:     $index = index - node.counter$ 
12:    return GETITEM( $child, index, val$ )
13:  end if
14: end function
```

---

Determining the index of a value involves the following  $O(p)$  algorithm shown in Algorithm 3, a significant improvement on the  $O(\log(n))$  search latency of array data structures.

---

**Algorithm 3** Retrieving Indices from Histogram

---

**Require:** Root of histogram tree,  $value$ **Ensure:** Index of target value

```
1: function GETINDEX( $root, value, defaultIdx$ )
2:    $node \leftarrow root$ 
3:   if  $node$  is null then
4:     return  $defaultIdx$ 
5:   end if
6:   if  $node.cnt \leq targetIndex$  then
7:      $child, idx = node.left, defaultIdx$ 
8:   else
9:      $child, idx = node.right, defaultIdx + node.cnt$ 
10:  end if
11:  return GETINDEX( $child, value, idx$ )
12: end function
```

---

#### F. Bandwidth Efficiency

1) *Computable Pointer Optimization:* The histogram nodes are optimized for bandwidth usage by employing a format that stores nodes without identifier information, achieved

by defining a subset of histogram nodes at configurable maximum depth  $L_c = 32$  with identifiers determined by memory location, eliminating the need for explicit identifiers of size  $w_i = O(\frac{p}{\lceil \log(n) \rceil})$  and pointers of size  $w_r = O(\log(n))$ , enabling the sparse section of the histogram to be updated bandwidth-efficiently using only counter variables of size  $w_{c,l} = O(\lceil \log(n) - l \rceil)$  for level  $l \in \{0, \log(n)\}$  as shown in Algorithm 4.

---

**Algorithm 4** New Node Creation and Resolution

---

**Require:** Level and counter value  $l$ , counter

```

1: function NODEORTUPLE( $l$ , counter)
2:    $node \leftarrow \text{root}$ 
3:   if  $l \leq L_c$  then ▷ Write to computable location
4:     return {counter : counter}
5:   else ▷ Allocate a new node and return its pointer
6:     return NEWNODE(counter)
7:   end if
8: end function

```

---

### G. FractalSortCPU Complexity

1) *Complexity:* The sort complexity where  $p$  is the precision,  $n$  is the number of inputs in a batch, and  $n_L = \frac{W}{w_c}$  is the number of  $W$ -bit registers needed to store  $p$  counters of  $w_c \approx \frac{\log(n)}{2}$  size along the key path, is approximately  $O(n \lceil \frac{p}{n_L} \rceil)$  serial and  $O(2\min(\lceil \frac{p}{n_L} \rceil, 2\log(n))) = O(2\min(\lceil \frac{pw_c}{W} \rceil, 2\log(n)))$  complexity since each key has to update its associated path in the histogram, which represents either a sparse trie with depth  $L_n = \log(n)$  or full histogram with depth  $L = p$  as shown in Table I.

2) *Memory usage:* The storage space utilized by the algorithm is determined by the size of the sparse histogram

$$O\left(\begin{cases} nw_c, & \text{if } n \leq 2^p \\ 2^p w_c, & \text{if } n > 2^p \end{cases}\right)$$

| Complexity Comparison            |                                            |                  |                                |                         |
|----------------------------------|--------------------------------------------|------------------|--------------------------------|-------------------------|
| HRS                              | GPU<br>Terasort<br>&<br>Bitonic<br>Network | Paradis          | Bonsai                         | FractalSortCPU          |
| $O(\lceil \frac{k}{d} \rceil n)$ | $O(\log^2 n)$                              | $O(\frac{n}{p})$ | $O(n \lceil \log_l(n) \rceil)$ | $O(2\min(p, 2\log(n)))$ |

TABLE I  
FRACTALSORTCPU COMPLEXITY SUMMARY [17] [6] [10]

3) *FractalSortCPUA Algorithm:* The FractalSortCPUA algorithm produces a sorted array of keys from the histogram in a bandwidth-efficient streaming operation shown in Algorithm 5. The algorithm iterates over all  $2^{l_n - l_b}$  bins in tree-walk order for trie depth  $l_n = L_n$  and bin width

---

**Algorithm 5** FractalSortCPUA: Sorted Array Reconstruction

---

**Require:** Entries  $E$ , index array  $I$ , bin counts  $C$ , precision  $p$ , tree depth  $l_n$ , bin depth  $l_b$

**Ensure:** Sorted array  $K$  of  $n$  reconstructed  $p$ -bit keys

```

1: function RECONSTRUCTALL( $E, I, C, p, l_n, l_b$ )
2:    $n_{bins} \leftarrow 2^{l_n - l_b}$ ,  $t \leftarrow p - l_n$ 
3:    $offset \leftarrow 0$ ,  $idx \leftarrow 0$ 
4:   for  $b = 0$  to  $n_{bins} - 1$  do
5:     if  $C[b] = 0$  then continue
6:   end if
7:    $keylow \leftarrow \text{BITREVERSE}(b, l_n - l_b)$ 
8:   for  $s = 0$  to  $C[b] - 1$  do
9:      $entry \leftarrow E[offset + I[offset + s]]$ 
10:     $trailing \leftarrow entry \& (2^t - 1)$ 
11:     $treeoff \leftarrow entry \gg t$ 
12:     $offval \leftarrow \text{BITREVERSE}(treeoff, l_b)$ 
13:     $K[idx] \leftarrow keylow \mid (offval \ll (l_n - l_b)) \mid (trailing \ll l_n)$ 
14:     $idx \leftarrow idx + 1$ 
15:   end for
16:    $offset \leftarrow offset + C[b]$ 
17: end for
18: return  $K$ 
19: end function

```

---

$l_b = \log(n_{bins})$ . The last level of the histogram contains the counter variable associated with each bin  $C$ . For each bin  $b$ , the bin identifier bits are recovered by reversing the tree ordering of the bin index. Within each bin, the sorted entries are traversed using the index array which maps each sorted position to an arrival position. Each entry is decoded by extracting the trailing bits and tree-reversed offset, and the full  $p$ -bit key is reconstructed by combining the bin identifier bits, offset bits and trailing bits.

The outer loop is  $O(n_{bins})$  and the inner loop process  $n$  entries exactly once with  $O(1)$  work per entry (index lookup, bit extraction, bit-reverse via LUT, and concatenation), giving total complexity  $O(n)$ . Memory traffic is one sequential read of the index array and one indexed read of the entry array, totaling  $\approx 2 \times \frac{p}{8}$  bytes per key. The FractalSortCPUA algorithm demonstrates 41% bandwidth efficiency as shown in Figure 10.

## IV. EVALUATION

The experiments are performed on a Windows 11 Pro device with an Intel(R) Core(TM) i7-8665U CPU at 1.90GHz capable of 2.11GHz processing with 32 GB RAM, 950 GB SK Hynix SSD storage and Intel Emerald Rapids 32 vCPU Debian 120GB RAM host for benchmarking [53]. The runtime environment includes 64 bit Python 3.9.1, and the tests involve comparison of FractalSortCPU to state-of-the-art CPU-based sorting algorithms baseline implemented in



Numba: quick sort, merge sort, heap sort, Tim sort, and a custom in-place implementation of radix sort with each reported result a statistical average of 10 runs [51]. Analysis of the effect of tuning batch size on latency and memory usage is performed on different data set sizes. Comparison of FractalSortCPU to the state of the art involves data sets of  $n \in \{2^{10} \rightarrow 2^{31}\}$ .

### A. FractalSortCPU Experiment Test Bed

The experiment setup includes a random number generator which populates batches of keys loaded to CPU and processed as shown in Figure 2.

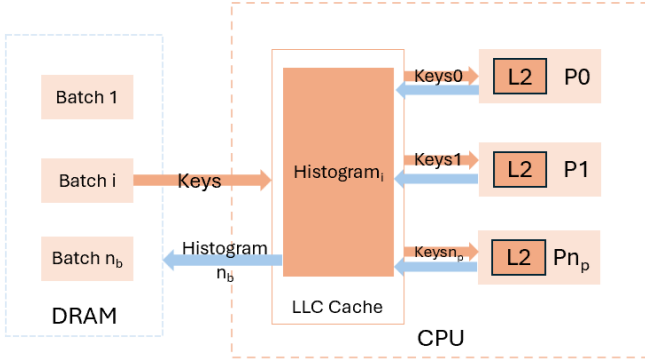


Fig. 2. FractalSortCPU Multi-batch Test Bed  $p = 16$

Although the keys are uniformly distributed, the histogram tree structure provides some robustness to skew since each key independently walks its own root-to-leaf path with no input bucketing. However, the counter width tapering scheme assumes approximately balanced subtrees where counter widths decrease as  $w_{c,l} = O(\lceil \log(n) - l \rceil)$ . Under heavy skew (e.g., Zipfian), subtree sizes may deviate from this assumption, and empirical evaluation across non-uniform distributions is left to future work.

### B. Latency Analysis

The latency curve for FractalSortCPU has sub-linear growth compared to the baselines and is 34% faster than the next best radix sort at  $n = 2^{29}$  as shown in Figure 3 and Table II. FractalSortCPU demonstrates lowest latency at  $n \geq 2^{21}$ . This speed-up is due to the reduced number of radix passes on compressed entries, with the bandwidth savings from histogram compression becoming apparent for large  $n$ .

| Algorithm    | Quick sort         | Merge sort         | Heap sort | Radix sort         | Fractal sort       |
|--------------|--------------------|--------------------|-----------|--------------------|--------------------|
| Latency (s)  | 59.51              | 80.36              | —         | 8.38               | 5.53               |
| Memory (MiB) | $4.10 \times 10^3$ | $4.10 \times 10^3$ | —         | $6.14 \times 10^3$ | $4.61 \times 10^3$ |

TABLE II  
SORT LATENCY (SECONDS) AND MEMORY USAGE (MiB) FOR  $n = 2^{29}, p = 32$  ON CPU WITH NUMBA

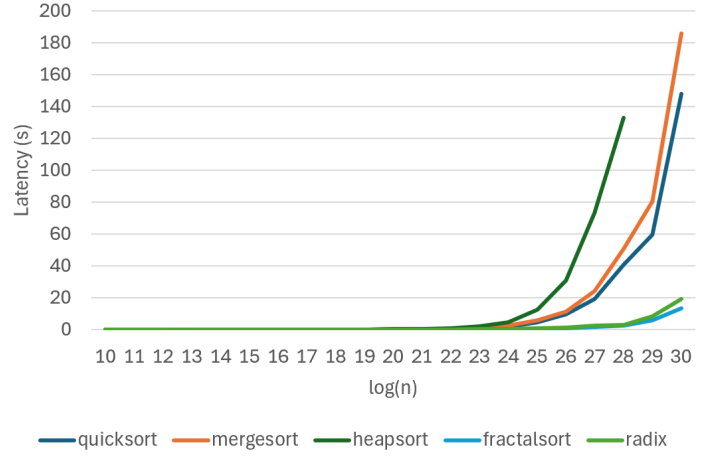


Fig. 5. FractalSortCPU Latency for  $n = 2^{30}, p = 32$

### C. Memory Usage Analysis

The memory usage analysis shows a 7x higher footprint for FractalSortCPU for small data sets which remains approximately constant for  $n \leq 2^{15}$ , at which point radix sort has higher utilization. This is because the histogram size is dependent on  $\log(n)$ , and due to the histogram cache between batch updates creating a trade-off between memory and latency as shown in Figure 6. At  $n = 2^{29}$  FractalSort consumes 4,608 MiB ranking second after the in-place comparison sorts (keys  $4n$  + entry array  $4n$  + radix temp buffers  $\approx n$  bytes), and 25% less than radix sort at 6,144 MiB. At  $n = 2^{30}$  FractalSort uses 9,216 MiB, maintaining the 25% memory advantage over radix sort as  $n$  increases.

### D. Batch Size Optimization

Streaming the dataset in input order in batches of equal size for more efficient processing with respect to the sorting resources available demonstrates varying benefits regarding latency and memory utilization depending on how the batches are processed both serially and in parallel as shown in Figure 3 and Figure 4.

1) *Parallel Batch Processing*: Processing batches in parallel involves dedicating a thread to each batch on the limited CPU thread pool. Increasing the number of batches has minimal impact on latency and memory usage for most data sets with latency standard deviation

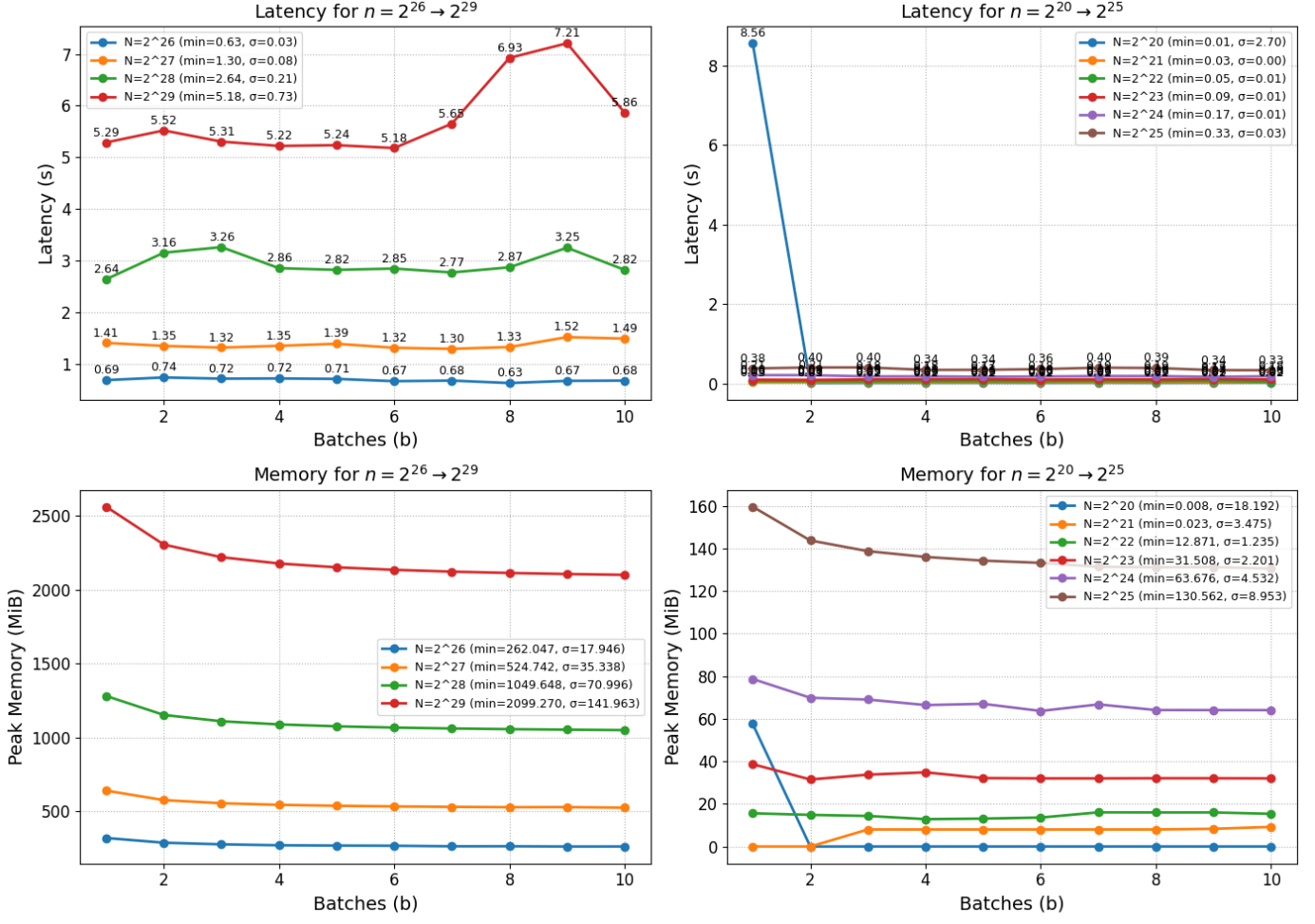


Fig. 3. FractalSortCPU Latency and Memory for Number of Parallel Batches  $b \in 1 \rightarrow 10$

$\sigma \in 0.03s \rightarrow 0.38s \approx 6\% \rightarrow 12\%$  and memory standard deviation  $\sigma \in 3 \times 10^{-6} \rightarrow 14\%$ .

An exception is  $n = 2^{25}$  and  $n = 2^{20}$  which show significant 20% reduction in latency at  $b = 6$  and 20KB reduction in memory  $b = 2$  respectively, as shown in Figure 3.

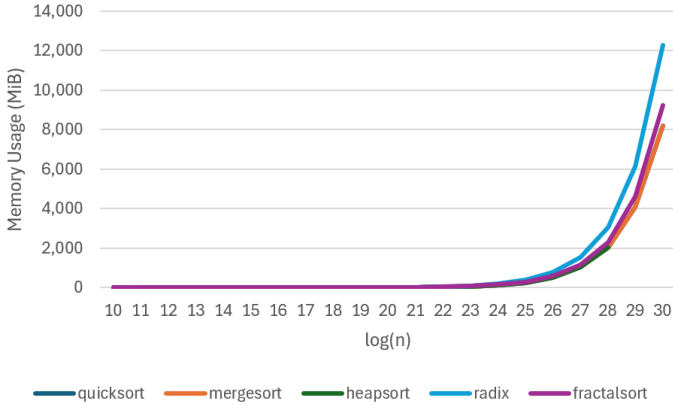


Fig. 6. FractalSortCPU Memory Usage for  $n = 2^{10} \rightarrow 2^{30}$  at  $p = 32$

2) *Serial Batch Processing*: Processing batches in serial is simpler with no thread creation overhead leading to lower latency for most data sets. Increasing the number of batches has moderate impact on latency for  $b \geq 2$ , with latency standard deviation  $\sigma = 0.54s$  across  $b \in 1 \rightarrow 20$  at  $n = 2^{29}$ . Latency at  $b = 1$  is higher due to larger per-bin radix sort temporary buffers, dropping from 7.63s to 5.35s at  $b = 2$  for all  $n \neq 2^{28}$ , averaging  $\approx 5.6s$  for  $b \geq 2$  with periodic variation as shown in Figure 7. Memory decreases monotonically from 2,612 MiB at  $b = 1$  to 2,074 MiB at  $b = 20$ , a 20% reduction, with diminishing returns beyond  $b = 10$  as shown in Figure 8.

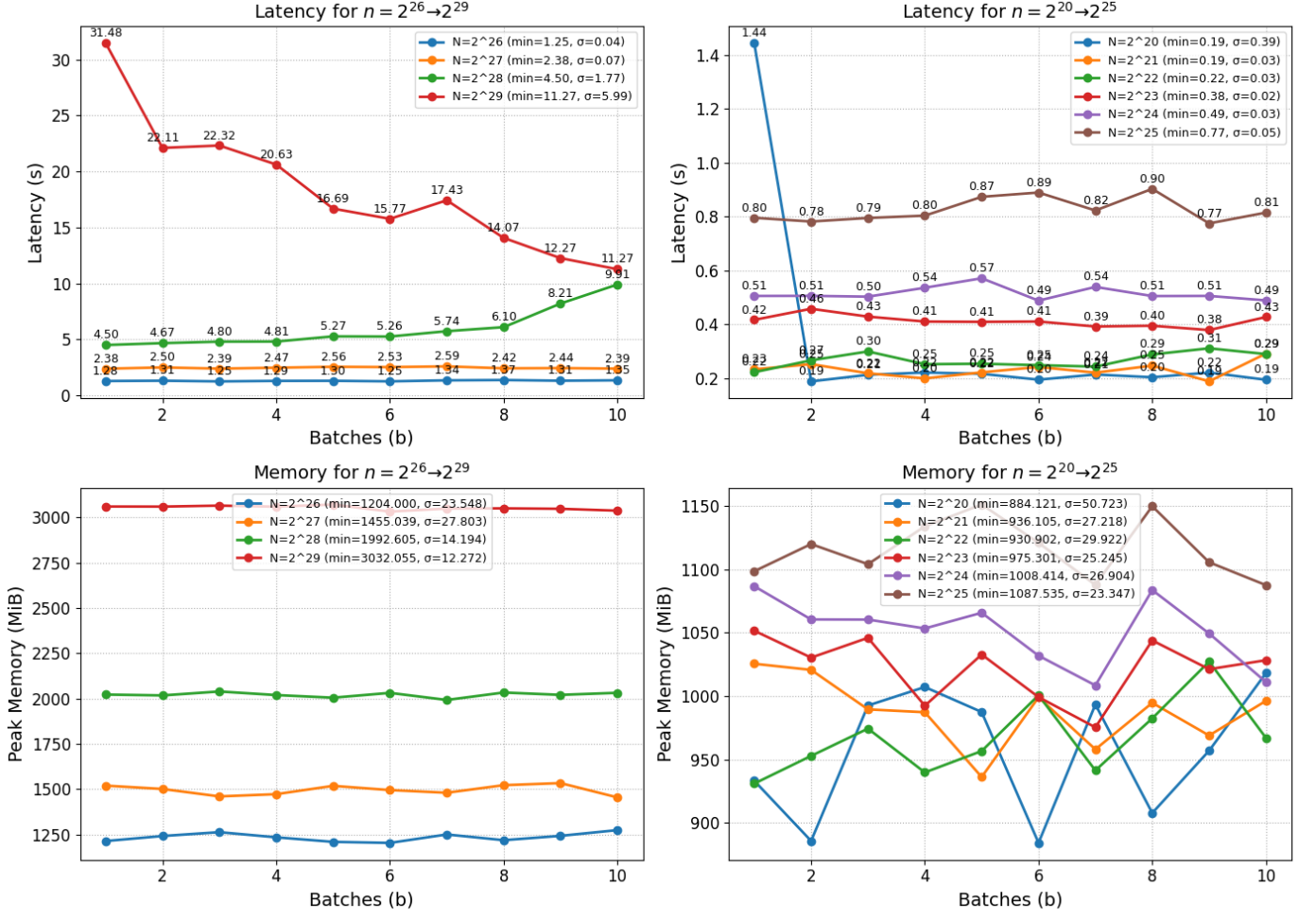


Fig. 4. FractalSortCPU Latency and Memory for Number of Serial Batches  $b \in 1 \rightarrow 10$

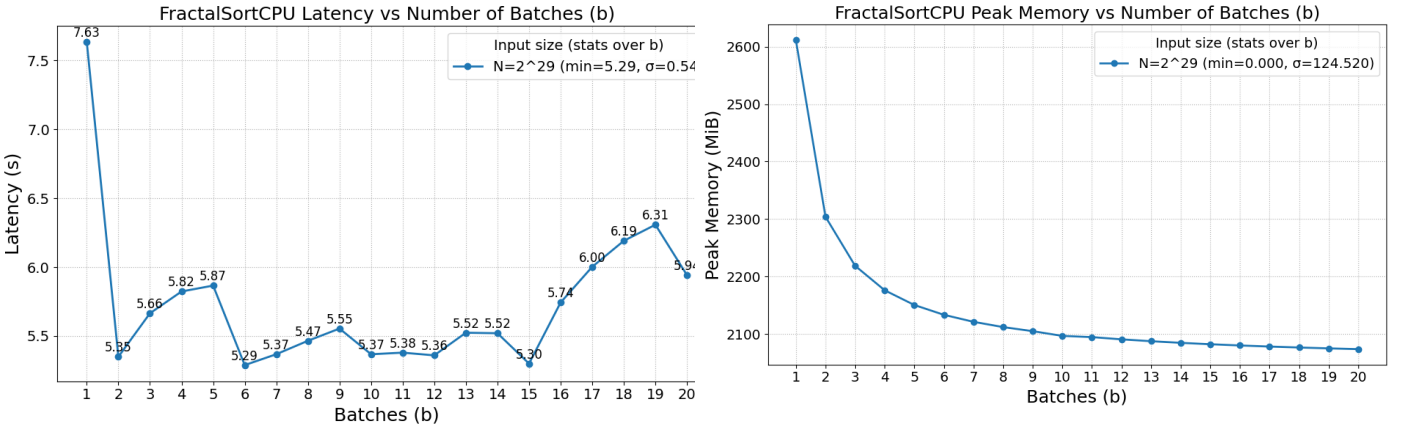


Fig. 7. FractalSortCPU Latency for  $n = 2^{29}$  and Serial Batch  $b \in 1 \rightarrow 20$

Fig. 8. FractalSortCPU Memory Usage for  $n = 2^{29}$  and Serial Batch  $b \in 1 \rightarrow 20$



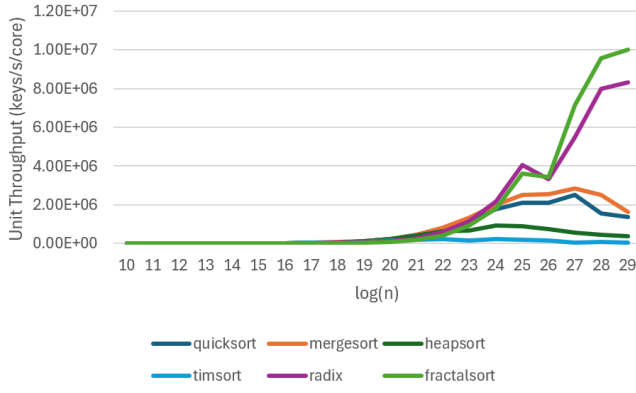


Fig. 9. Unit Throughput for  $2^{10} \leq n \leq 2^{30}$

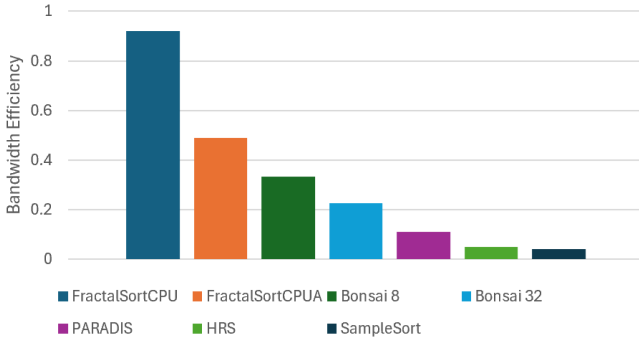


Fig. 10. Bandwidth Efficiency for 16GB datasets ( $p = 16$ ) compared to Bonsai [10], Paradis [6], HRS [17] and SampleSort [23]

### E. Unit Throughput

The unit throughput is calculated as  $T_{unit} = \frac{n}{t \times n_c}$  where  $t$  is the latency and  $n_c$  is the number of CPU cores (4 cores on 8 threads). FractalSortCPU has the highest unit throughput at  $n \geq 2^{26}$  growing to  $1 \times 10^7$  keys/s, a 20% speed up over the second best radix sort at  $n = 2^{30}$  at  $p = 32$  as shown in Figure 9. State-of-the-art CPU-based Paradis [6] sorts 4GB of  $p = 16$  data in 4.6s on a 32-core Intel Xeon (E7-8837) processor for unit throughput  $14.59 \times 10^6$  keys/s, while FractalSortCPU latency is 21.22s at  $b = 14$  for  $25.30 \times 10^6$  keys/s, a 73.4% speed up.

### F. FractalSortCPU on Large Datasets

For large data sets  $n \geq 2^{30}$  FractalSortCPU exhibits low latency  $t_{2^{30}} = 13.4s$  in contrast to the comparison sorts which demonstrate superlinear growth as shown in Figure 5. Memory usage increases linearly with  $n$  at  $9n$  bytes, 25% lower than radix sort at  $12n$  bytes as shown in Figure 6. The better performance of FractalSortCPU at large  $n$  is due to its latency and memory usage being primarily determined by histogram size which grows sublinearly in  $n$  as  $O(2^p \log_2(n))$ , determined by the counter variable width  $\approx \log_2(n)$  stored in each node as shown in Figure 6.

1) *Bandwidth Efficiency*: Following the bandwidth-efficiency analysis used in prior sorting work [17] [54], we define bandwidth efficiency as the ratio of actual throughput to total memory (DRAM) bandwidth consumed:

$$b_{eff} = \frac{T_{actual}}{B_{DRAM}} \quad (1)$$

where  $T_{actual}$  is the sum of read and write throughput in bytes per second for  $n$  keys of  $p$  bits sorted in time  $t$ , and  $B_{DRAM}$  is the total DRAM traffic in bytes per second including all intermediate reads and writes. The ratio is dimensionless with higher values indicating more efficient use of memory bandwidth. Algorithms that require multiple passes over the data or store large intermediate structures have lower  $b_{eff}$  due to increased  $B_{DRAM}$ .

FractalSortCPU is compared to state-of-the-art algorithms over a variety of data set sizes to explore on-chip and off-chip memory access and sort performance. The bandwidth efficiency of FractalSortCPU is significantly higher than the state of the art due to the small size of the compressed histogram which fits entirely on CPU LLC cache at  $p = 16$ . Compared to state-of-the-art sorting algorithms, FractalSortCPU performs very favorably with a 6x increase in bandwidth efficiency compared to CPU [6], 3x compared to GPU [17] and 2.5x compared to FPGA [10] for 16GB data sets as shown in Figure 10.

## V. CONCLUSION AND RECOMMENDATIONS

FractalSortCPU introduces a histogram compression scheme demonstrating low latency and sublinear memory growth as dataset size increases. It performs better for large  $n$ , hence suited for large dataset processing where latency and memory usage are key concerns. Though in-place radix sort shows superior performance for medium datasets and comparison-based sorters for small datasets, FractalSortCPU scales more slowly for large datasets with 25% lower memory usage compared to radix sort at  $n = 2^{29}$ .

The key contributions of FractalSortCPU include a space-efficient fractal sorting algorithm, involving a precision-depth bound histogram tree of sparse counters, and a parallel merge operation that eliminates the need for data pre-processing and input partitioning. Caching of the histogram allows the sharing of merge tree between batches enabling the amortization of batch-switching costs as the number of batches increases with a 20% reduction in batch memory usage at  $b = 20$ .

FractalSortCPU provides significant improvement over state-of-the-art sorting algorithms on CPU, GPU, and FPGA by 6x, 3x, and 2.5x in bandwidth efficiency on 512MB to 32GB data sets, a key benefit of using this algorithm, and demonstrates a 73.4% improvement in unit throughput compared to the state-of-the-art CPU algorithm Paradis as shown in Figure 9. This is because no input partitioning

is needed which eliminates round trips to memory storage required during pre-processing, achieving a 3x improvement in bandwidth efficiency over state-of-the-art Bonsai on 16KB-32GB data sets as shown in Figure 10. Optimizing the count variable width  $w_c$  based on the reduction of subtree size with increasing merge tree depth improves bandwidth efficiency and data compression.

FractalSortCPU is useful for sorting data streaming scenarios, and implementation on CPU enables both significant performance improvements and greater accessibility due to lower adoption costs. Together with the FPGA implementation [54], the FractalSort family provides bandwidth-efficient sorting across heterogeneous platforms. Future work will involve exploration of the GPU platform for higher parallelism and data access rates and exploration of novel high-precision sorting optimizations.

## REFERENCES

- [1] B. G. R. R. B. and P. S. Yu, "CellSort: High Performance Sorting on the Cell Processor." In: *Proc. VLDB* (2007), pp. 1286–1297.
- [2] R. M. J. T. G. Alonso, "Sorting networks on FPGAs." In: *VLDB* 21 (2012), pp. 1–23. <https://doi.org/10.1007/s00778-011-0232-z>.
- [3] S. J. S. X. R. Arvind, "Terabyte sort on FPGA-accelerated flash storage." In: *International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (2017).
- [4] S. J. S. X. S. Arvind, "Terabyte Sort on FPGA-Accelerated Flash Storage." In: *Proceedings of the 25th IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (2017).
- [5] F. Berthelot, F. Nouvel, and D. Houzet, "Partial and dynamic reconfiguration of FPGAs: a top down design methodology for an automatic implementation." In: *Proceedings 20th IEEE International Parallel Distributed Processing Symposium* (2006), 4 pp. DOI: 10.1109/IPDPS.2006.1639466.
- [6] M. C. D. B. R. B. Bordawekar, "PARADIS: An efficient parallel algorithm for in-place radix sort." In: *Very Large Data Bases (VLDB)* (2015).
- [7] M. A. R. X. C. L. Y. H. D. L. J. Bradley, "Spark SQL: Relational data processing in spark." In: *International Conference on Management of Data* (2015). URL: <https://sortbenchmark.org/>.
- [8] L. Canese et al., "A Novel Digital Equalizer Based on RF Sampling Beyond GHz." In: *IEEE Access* 12 (2024), pp. 92560–92572. DOI: 10.1109/ACCESS.2024.3422802.
- [9] J. Chhugani et al., "Efficient implementation of sorting on multi-core SIMD CPU architecture." In: *Proceedings of the VLDB Endowment* 1(2) (2008), pp. 1313–1324.
- [10] N. S. W. Q. V. A. M. C. J. Cong, "Bonsai: High-Performance Adaptive Merge Tree Sorting." In: *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)* (2020).
- [11] W. Q. L. G. M. C. J. Cong, "FANS: FPGA-accelerated near-storage sorting." In: *Proceedings of the 29th IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (2021).
- [12] W. Q. L. G. Z. F. M. C. J. Cong, "TopSort: A High-Performance Two-Phase Sorting Accelerator Optimized on HBM-based FPGAs." In: *IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (2022). DOI: <https://doi.org/10.1109/FCCM53951.2022.9786209>.
- [13] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters." In: *Communications of the ACM* 51(1) (Jan. 2008), pp. 107–113. DOI: 10.1145/1327452.1327492. URL: <https://doi.org/10.1145/1327452.1327492>.
- [14] A. Dutta and M. Saha, "Contrasting Parallelized with Sequential Sorting." In: *2022 IEEE 7th International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*. Vol. 7. 2022, pp. 256–259. DOI: 10.1109/ICRAIE56454.2022.10054300.
- [15] G. Graefe, "Implementing sorting in database systems." In: *ACM Computing Surveys* 38(3) (Sept. 2006), 10–es. DOI: 10.1145/1132960.1132964. URL: <https://doi.org/10.1145/1132960.1132964>.
- [16] J. Gray, "Sort Benchmark Home Page." (2023). URL: <https://sortbenchmark.org/>.
- [17] E. S. H. A. Jacobsen, "A memory bandwidth-efficient hybrid radix sort on GPUs." In: *International Conference on Management of Data (SIGMOD)* (2017). DOI: <https://doi.org/10.1145/3035918.3064043>.
- [18] D. Koch and J. Torresen, "FPGASort: A high performance sorting architecture exploiting run-time reconfiguration on FPGAs for large problem sorting." In: *Proceedings of the 19th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (2011), pp. 45–54.
- [19] C. G. Z. B. P. Laskov, "FPGA vs. Multi-core CPUs vs. GPUs: Hands-On Experience with a Sorting Application." In: *Lecture Notes in Computer Science* 6310 (2017), pp. 105–117.
- [20] X. Luo et al., "A Memory-Disaggregated Radix Tree." In: *ACM Transactions on Storage* 20(3) (June 2024). DOI: 10.1145/3664289. URL: <https://doi.org/10.1145/3664289>.
- [21] T. Maltenberger et al., "Evaluating multi-GPU sorting with modern interconnects." In: *Proceedings of the 2022 International Conference on Management of Data* (2022), pp. 1795–1809.
- [22] D. Merrill and A. Grimshaw, "High Performance and Scalable Radix Sorting: A Case Study of Implementing Dynamic Parallelism for GPU Computing." In: *Parallel Processing Letters* 21(02) (2011), pp. 245–272. DOI: 10.1142/S0129626411000187. URL: <https://doi.org/10.1142/S0129626411000187>.
- [23] H. C. S. M. M. F. P. Milder, "FPGA-Accelerated Samplesort for Large Data Sets." In: *Proceedings of the 28th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)* (2020).
- [24] P. Papaphilippou, C. Brooks, and W. Luk, "FLiMS: Fast lightweight merge sorter." In: *2018 International Conference on Field-Programmable Technology (FPT)*. IEEE, 2018, pp. 78–85.
- [25] N. Satish, M. Harris, and M. Garland, "Designing efficient sorting algorithms for manycore GPUs." In: *2009 IEEE International Symposium on Parallel & Distributed Processing*. IEEE, 2009, pp. 1–10.
- [26] N. Satish et al., "Fast sort on CPUs and GPUs: a case for bandwidth oblivious SIMD sort." In: *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data* (2010), pp. 351–362.
- [27] M. El-Sharkawy et al., "Re-configurable parallel Feed-Forward Neural Network implementation using FPGA." In: *Integration* 97 (2024), p. 102176. DOI: <https://doi.org/10.1016/j.vlsi.2024.102176>.
- [28] T. Usui, T. Van Chu, and K. Kise, "A cost-effective and scalable merge sorter tree on FPGAs." In: *2016 Fourth International Symposium on Computing and Networking (CANDAR)*. IEEE, 2016, pp. 47–56.
- [29] C. Zhang, R. Chen, and V. Prasanna, "High throughput large scale sorting on a CPU-FPGA heterogeneous platform." In: *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2016, pp. 148–155.
- [30] M. Zuluaga, P. Milder, and M. Püschel, "Streaming Sorting Networks." In: 21(4) (May 2016). DOI: 10.1145/2854150. URL: <https://doi.org/10.1145/2854150>.
- [31] S. Asano, T. Maruyama, and Y. Yamaguchi, "Performance comparison of FPGA, GPU and CPU in image processing." In: *2009 International Conference on Field Programmable Logic and Applications (FPL)* (2009), pp. 126–131. DOI: <https://doi.org/10.1109/FPL.2009.5272532>.
- [32] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [33] T. Bingmann, S. Sanders, and J. Singler, "Parallel string sample sort," *ACM Journal of Experimental Algorithmics*, vol. 23, no. 1, pp. 1–23, 2018.
- [34] G. E. Blelloch, C. E. Leiserson, B. M. Maggs, C. G. Plaxton, S. J. Smith, and M. Zagha, "A comparison of sorting algorithms for the connection machine CM-2," in *Proceedings of the 3rd ACM Symposium on Parallel Algorithms and Architectures*, 1991, pp. 3–16.
- [35] P. Sanders and J. L. Träff, "Parallel sorting in message-passing systems," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, 1997, pp. 1–10.
- [36] F. Dehne, A. Fabri, and A. Rau-Chaplin, "Scalable parallel geometric algorithms for coarse grained multicomputers," in *Proceedings of the IEEE Symposium on Parallel and Distributed Processing*, 1996, pp. 298–305.

- [37] D. Ajwani, R. Dementiev, and P. Sanders, “A cache-oblivious distribution sweeping framework for orthogonal range reporting and related problems,” *ACM Transactions on Algorithms*, vol. 12, no. 1, pp. 1–33, 2016.
- [38] T. Hagerup and C. Rüb, “Optimal merging and sorting on the EREW PRAM,” *Information Processing Letters*, vol. 33, no. 4, pp. 181–185, 1991.
- [39] N. Leischner, V. Osipov, and P. Sanders, “GPU sample sort,” in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, 2010, pp. 1–10.
- [40] S. Shi and X. Zhang, “An efficient parallel radix sort algorithm on multicore systems,” *Proceedings of the International Conference on Computer Science and Service System*, 2012, pp. 450–453.
- [41] Intel Corporation, “Intel Integrated Performance Primitives (Intel IPP) – Data Processing and Sorting,” 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/ipp.html>
- [42] H. Inoue, T. Moriyama, H. Komatsu, and T. Nakatani, “AA-Sort: A new parallel sorting algorithm for multi-core SIMD processors,” in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, 2007, pp. 189–198.
- [43] F. Strati, P. Elvinger, T. Kerimoglu, and A. Klimovic, “ML Training with Cloud GPU Shortages: Is Cross-Region the Answer?” In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys ’24)*, Association for Computing Machinery, New York, NY, USA, 107–116. <https://doi.org/10.1145/3642970.3655843>
- [44] M. Dang’ana, “FractalSortCPU Source Code,” 2025. [Online]. Available: [https://github.com/mikdangana/fractalsort\\_cpu/](https://github.com/mikdangana/fractalsort_cpu/)
- [45] “Numba reference,” <https://numba.pydata.org/>, 2025, accessed: 2025-08-04.
- [46] M. Axtmann, S. Witt, D. Ferizovic, and P. Sanders, “Engineering in-place (shared-memory) sorting algorithms,” *ACM Trans. Parallel Comput.*, vol. 9, no. 1, Jan. 2022. [Online]. Available: <https://doi.org/10.1145/3505286>.
- [47] Y. Li, “Performance analysis of efficient sorting algorithms in big data processing,” *Procedia Computer Science*, vol. 262, pp. 44–50, 2025.
- [48] N. S. Abuba, E. Y. Baagyere, C. I. Nakpih, and J. K. Wiredu, “Optiflex-sort: A hybrid sorting algorithm for efficient large-scale data processing,” *Journal of Advances in Mathematics and Computer Science*, vol. 40, no. 2, pp. 67–81, 2025.
- [49] J. Anantpur and R. Govindarajan, “Taming warp divergence,” in *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2017, pp. 50–60.
- [50] C. Shuang, J. Shunning, H. Bingsheng, and T. Xuenyan, “A study of sorting algorithms on approximate memory,” *San Francisco*, 2016.
- [51] “Numpy reference,” <https://numpy.org>, 2025, accessed: 2025-08-04.
- [52] A. Singhal, R. Van der Wijngaart, and P. Barry, “Atomic read modify write primitives for i/o devices,” Tech. rep, Tech. Rep., 2008.
- [53] “Intel 7-8665u cpu reference,” <https://www.intel.com/content/www/us/en/products/sku/193563/intel-core-i78665u-processor-8m-cache-up-to-4-80-ghz/specifications.html>, 2025, accessed: 2025-08-04.
- [54] M. Dang’ana and H.-A. Jacobsen, “FractalSort: High precision compressed radix sort on FPGA,” *IEEE Transactions on Computers*, vol. 75, no. 5, pp. 1752–1766, May 2026. DOI: 10.1109/TC.2026.3653702.
- [55] M. Dang’ana, Y. Zhang, and H.-A. Jacobsen, “Ksurf-Drone: Attention Kalman filter for contextual bandit optimization in cloud resource allocation,” *IEEE Transactions on Cloud Computing*, vol. PP, no. 99, pp. 1–15, Jan. 2026. DOI: 10.1109/TCC.2026.3653558.